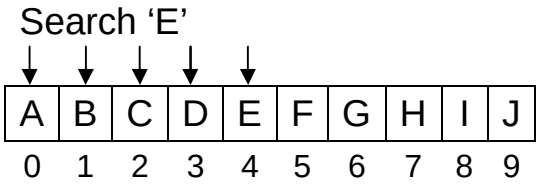
CORE – 14: Algorithm Design Techniques (Unit – 2)

Searching Algorithms are designed to check for an element or retrieve an element from any data structure where it is stored. Based on the type of search operation, these algorithms are generally classified into two categories:

1. Linear Search
2. Binary Search

Linear Search: A linear search is also known as a sequential search that simply scans each element at a time. Let's consider a simple example.

Suppose we have an array of 10 elements as shown in the figure. The figure shows an array of character type having 10 values.



If we want to search 'E', then the searching begins from the 0th element and scans each element until the element, i.e., 'E' is not found. We cannot directly jump from the 0th element to the 4th element, i.e., each element is scanned one by one till the element is not found.

Time Complexity of Linear search: As linear search scans each element one by one until the element is not found. If the number of elements increases, the number of elements to be scanned is also increased. We can say that the *time taken to search the elements is proportional to the number of elements*. Therefore, the worst-case complexity is **O (n).**

Algorithm to search a given element x in a given array arr[ ]

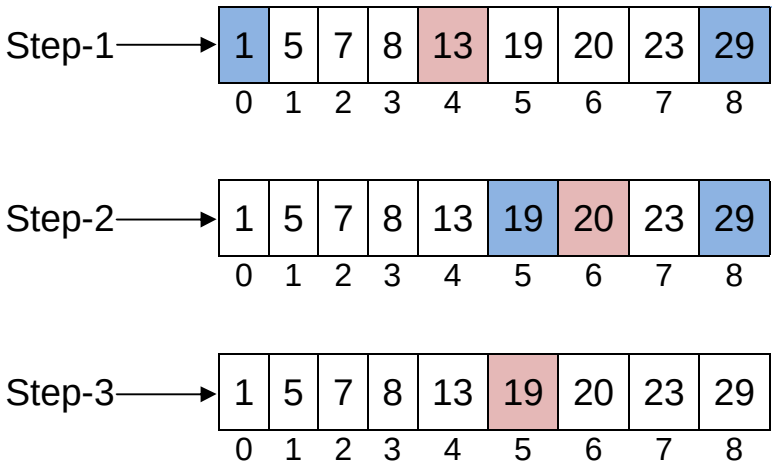
- Start from the leftmost element of arr[ ] and one by one compare x with each element of arr[ ]
- If x matches with an element, return the index.
- If x doesn't match with any of elements, return -1.

Binary Search:

1. In Binary Search technique, we search an element in a sorted array recursively by dividing the interval in half.
2. Firstly, we take the whole array as an interval.
3. If the Pivot Element (the item to be searched) is less than the item in the middle of the interval, we discard the second half of the list and recursively repeat the process for the first half of the list by calculating the new middle and last element.
4. If the Pivot Element (the item to be searched) is greater than the item in the middle of the interval, we discard the first half of the list and work recursively on the second half by calculating the new beginning and middle element.
5. Repeatedly, check until the value is found or interval is empty.

Example:

Item to be searched = 19



beg = 0, end = 8
mid = (beg + end)/2 = 4
a[mid] = 13 which is then 19
beg = mid + 1 = 5
mid = (beg + end)/2 = 13/2 = 6

a[mid] = 20 which is greater than 19
end = mid - 1 = 5
mid = (beg + end)/2 = 5

a[mid] = 19
return location = 5

ALGORITHM - BinarySearch(A, lower_bound, upper_bound, VAL)

Step 1: SET BEG = lower_bound, END = upper_bound, POS = - 1
Step 2: Repeat Steps 3 and 4 while BEG <= END
Step 3: SET MID = (BEG + END)/2
Step 4: IF A[MID] = VAL
SET POS = MID
Return POS
Go to Step 6
ELSE IF A[MID] > VAL
SET END = MID - 1
ELSE
SET BEG = MID + 1
[END OF IF]
Step 5: IF POS = -1
PRINT "VALUE IS NOT PRESENT IN THE ARRAY"
[END OF IF]
Step 6: EXIT

Time complexity of Binary Search: At each iteration, the array is divided by half. So let's take length of array be **n**

- At Iteration 1, Length of array = n
- At Iteration 2, Length of array = $n/2$
- At Iteration 3, Length of array = $(n/2)/2 = n/2^2$
- Therefore, after Iteration k, Length of array = $n/2^k$
- Also, we know that after k divisions, the length of array becomes 1
- Therefore $n/2^k = 1$
  $\Rightarrow n = 2^k$
- Applying log function on both sides, we get-
  $\log_2 (n) = \log_2 (2^k)$
  $\Rightarrow \log_2 (n) = k \log_2 (2)$
  $\Rightarrow k = \log_2 (n)$

Hence, the time complexity of Binary Search is ***$\log_2 (n)$***

Merge Sort: It closely follows the divide & Conquers paradigm. Conceptually, it works as follows:

1. **Divide:** Divide the unsorted list into two sub-lists of about half the size.
2. **Conquer:** Sort each of the two sub-lists recursively until we have list sizes of length 1, in which case the list items are returned.
3. **Combine:** Join the two sorted Sub lists back into one sorted list.

Example:

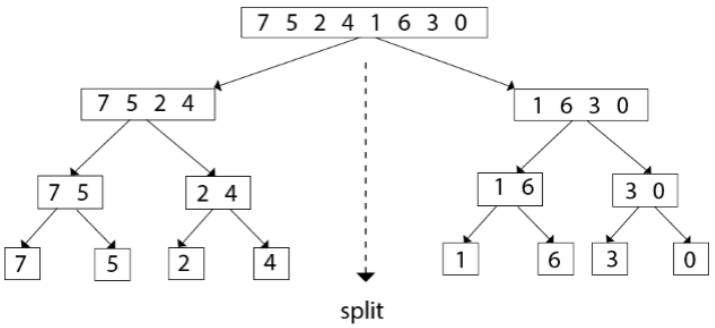


Figure: Merge sort divide phase

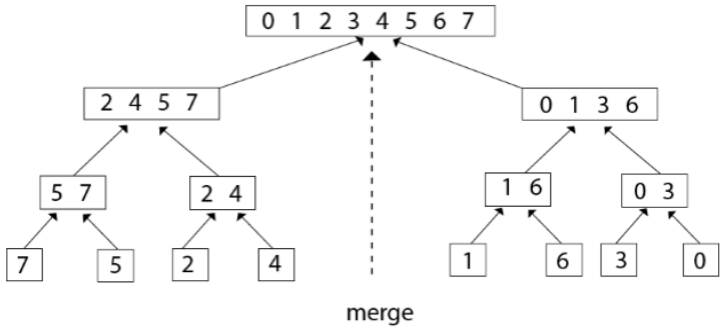


Fig: Merge sort: combine phase

```
/* l is for left index and r is right index of the sub-array of arr to be sorted */
void mergeSort(int arr[ ], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        // divide phase
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);        // combine phase
    }
}

void merge(int arr[ ], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;      int n2 = r - m;
    int L[n1], R[n2];        /* create temp arrays */

    for (i = 0; i < n1; i++)            /* Copy data to temp arrays L[ ] */
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)            /* Copy data to temp arrays R[ ] */
        R[j] = arr[m + 1 + j];

    /* Merge the temp arrays back into arr[l..r]*/
    i = 0; // Initial index of first sub-array
    j = 0; // Initial index of second su-barray
    k = l; // Initial index of merged subarray
```

```
    while (i < n1 && j < n2) {
        if (L[i] <= R[j])
                { arr[k] = L[i];     i++;  }
        else
                { arr[k] = R[j];     j++;  }
        k++;
    }

    while (i < n1)          /* Copy the remaining elements of L[ ], if there are any */
        { arr[k] = L[i];  i++;  k++;  }

    while (j < n2)          /* Copy the remaining elements of R[ ], if there are any */
        { arr[k] = R[j];  j++;  k++;  }
}
```

Analysis of Merge Sort: Let T (n) be the total time taken in Merge Sort

1. Sorting two halves will be taken at the most 2T(n/2) time
2. When we merge the sorted lists, we have a total n-1 comparison

So, the relational formula becomes; T(n) = 2T(n/2) + n-1
Ignoring the constant -1, it becomes; T(n) = 2T(n/2) + n
The above recurrence can be solved either using the Recurrence Tree method or the Master method. It falls in case-II of Master Method and the solution of the recurrence is θ(nLogn). So, the time complexity of Merge Sort is **θ(nLogn)** in all 3 cases (worst, average and best) as merge sort always divides the array into two halves and takes linear time to merge two halves.

Quick Sort: Like Merge Sort, Quick Sort is a Divide and Conquer algorithm. It picks an element as pivot and partitions the given array in such a way that all the elements which are equal or smaller than the pivot are put to the left of the pivot and all the elements which are greater than the pivot are put to the right of pivot. Then, it recursively sort each sub-array. There are many different versions of quick sort that pick pivot in different ways.

1. Always pick last element as pivot (implemented below)
2. Always pick first element as pivot.
3. Pick a random element as pivot.
4. Pick median as pivot.

Illustration of partition() :

arr[ ] =

10	80	30	90	40	50	70
0	1	2	3	4	5	6

Here, low = 0, high = 6 and pi = 70, i = low-1 = -1

For j = 0, arr[ ] =

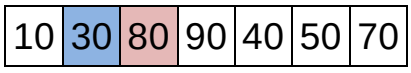
10	80	30	90	40	50	70
0	1	2	3	4	5	6

//since arr[j] <= pivot, do i++, now i = 0, and swap(arr[i], arr[j]), no change as i = j

For j = 1, arr[ ] =

10	80	30	90	40	50	70
0	1	2	3	4	5	6

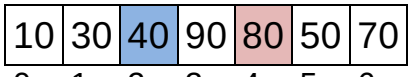
// do nothing since arr[j] > pivot

For j = 2, arr[ ] = 

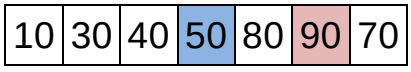
// Since arr[j] <= pivot, do i++, now i = 1 and swap(arr[i], arr[j])

For j = 3, arr[ ] = 

// do nothing since arr[j] > pivot

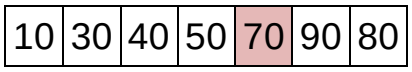
For j = 4, arr[ ] = 

// Since arr[j] <= pivot, do i++, now i = 2 and swap(arr[i], arr[j])

For j = 5, arr[ ] = 

// Since arr[j] <= pivot, do i++, now i = 3 and swap(arr[i], arr[j])

We come out of loop because j is now equal to high-1. Finally we place pivot at correct position by swapping arr[i+1] and arr[high] (or pivot). So, the array is now;

arr[ ] = 

// 80 and 70 Swapped

Now, the pivot element 70 is at its correct place. All elements smaller than 70 are before it and all elements greater than 70 are after it.

Pseudo Code for recursive quickSort function :

```
quickSort(arr[ ], low, high) {          /* low = Starting index,  high = Ending index */
    if (low < high) {
        pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);  // Before pi
        quickSort(arr, pi + 1, high); // After pi
    }
}
```

Pseudo code for partition()

```
partition (arr[ ], low, high) {
    pivot = arr[high];
    i = (low - 1)  // Index of smaller element
    for (j = low; j <= high- 1; j++) {
        if (arr[j] <= pivot) {        // If current element is smaller than the pivot
            i++;    // increment index of smaller element
            swap arr[i] and arr[j]
        }
    }
    swap arr[i + 1] and arr[high]) // put the pivot at i+1 index
    return (i + 1)
}
```

Analysis of Quick Sort: Time taken by Quick Sort in general can be written as;

$$T(n) = T(k) + T(n-k-1) + \theta(n)$$

The first two terms are for two recursive calls, the last term is for the partition process. k is the number of elements which are smaller than pivot. The time taken by Quick Sort depends upon the input array and partition strategy. Following are three cases.

Worst Case: The worst case occurs when the partition process always picks greatest or smallest element as pivot. If we consider above partition strategy where last element is always picked as pivot, the worst case would occur when the array is already sorted in increasing or decreasing order. Following is recurrence for worst case.

$T(n) = T(0) + T(n-1) + \theta(n)$ which is equivalent to $T(n) = T(n-1) + \theta(n)$

The solution of above recurrence is **$\theta(n^2)$.**

Best Case: The best case occurs when the partition process always picks the middle element as pivot. Following is recurrence for best case.

$T(n) = 2T(n/2) + \theta(n)$

The solution of above recurrence is **(nLogn).** It can be solved using case 2 of Master Theorem. *The Average Case time complexity of quick sort is also* **O(nLogn)**

Heap Sort: Following are the Properties of a heap:

- Heaps are complete binary trees implemented using an array.
- Parent of a node i is $\lfloor i/2 \rfloor$
- Left child of the node i is $2*i$
- Right child of the node i is $(2*i)+1$
- Max-heap property → The value of a node is greater than or equal to the value of its children i.e., $A[Parent[i]] \geq A[i]$ for all nodes $i>1$.
- Min-heap property → The value of a node is either smaller or equal to the value of its children i.e., $A[Parent[i]] \leq A[i]$ for all nodes $i>1$.
- HEAPIFY is a function used to maintain the heap property of any heap. It is applied to a node when the children of the node are following the property of a heap but the node itself may be violating it. It runs in O(lgn) time.
- BUILD-HEAP is a function used to make a heap from an array. It runs in O(n) time.

Working of Heap Sort: Heap sort is implemented using a max-heap. In a max-heap, the maximum element is at the root of the tree and is also the first element of the array i.e., A[1].

Step by Step Process: The Heap sort algorithm to arrange a list of elements in ascending order is performed using following steps...

- Step 1 - Construct a Binary Tree with given list of Elements.
- Step 2 - Transform the Binary Tree into Max Heap.
- Step 3 - Delete the root element from Max Heap using Heapify method.
- Step 4 - Put the deleted element into the Sorted list (i.e. at the last position).

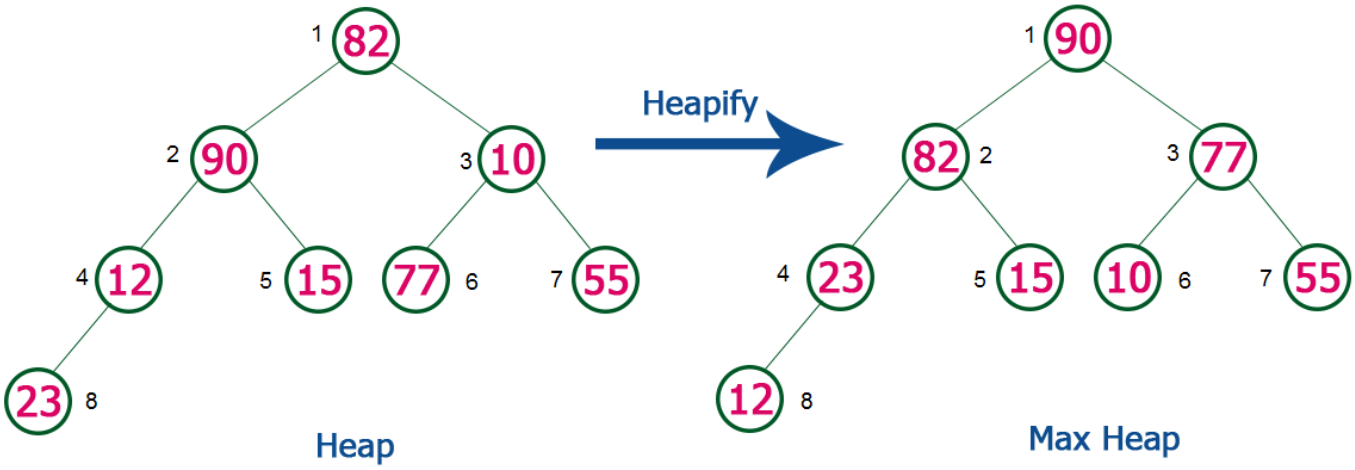
- Step 5 - Decrease the Heap size by 1.
- Step 6 - Repeat the same until Min Heap becomes empty.
- Step 7 - Display the sorted list.

Example:

Consider the following list of unsorted numbers which are to be sort using Heap Sort

82, 90, 10, 12, 15, 77, 55, 23

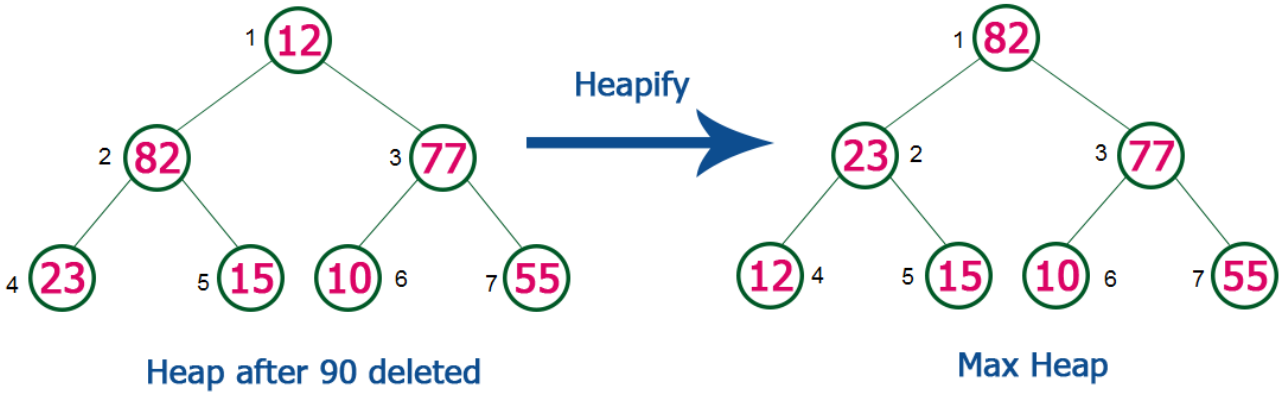
Step 1 - Construct a Heap with given list of unsorted numbers and convert to Max Heap



list of numbers after heap converted to Max Heap

90, 82, 77, 23, 15, 10, 55, 12

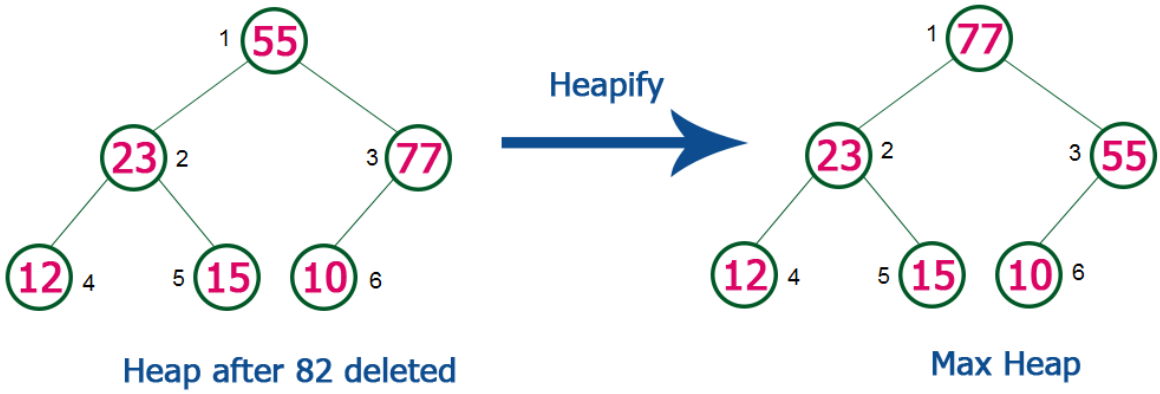
Step 2 - Delete root (**90**) from the Max Heap. To delete root node it needs to be swapped with last node (**12**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 90 with 12.

12, 82, 77, 23, 15, 10, 55, **90**

Step 3 - Delete root (**82**) from the Max Heap. To delete root node it needs to be swapped with last node (**55**). After delete tree needs to be heapify to make it Max Heap.



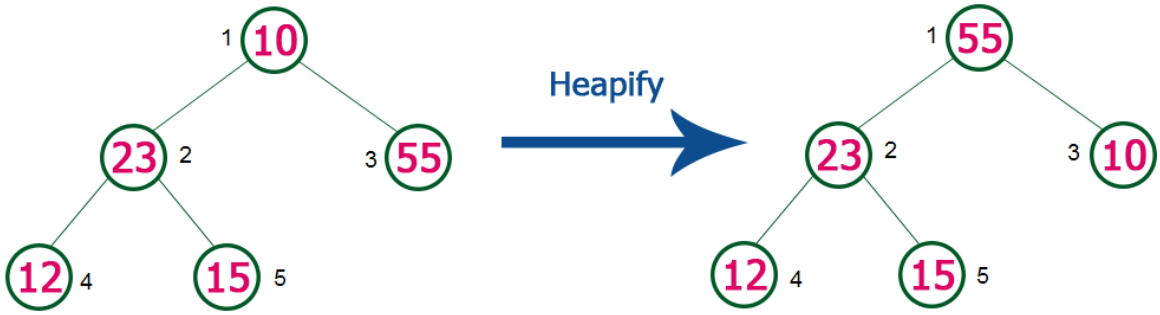
Heap after 82 deleted

Max Heap

list of numbers after swapping 82 with 55.

12, 55, 77, 23, 15, 10, **82**, **90**

Step 4 - Delete root (**77**) from the Max Heap. To delete root node it needs to be swapped with last node (**10**). After delete tree needs to be heapify to make it Max Heap.



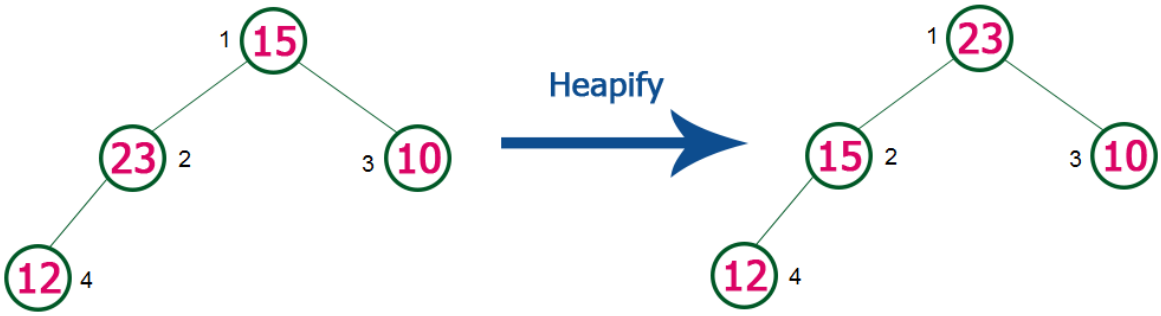
Heap after 77 deleted

Max Heap

list of numbers after swapping 77 with 10.

12, 55, 10, 23, 15, **77**, **82**, **90**

Step 5 - Delete root (**55**) from the Max Heap. To delete root node it needs to be swapped with last node (**15**). After delete tree needs to be heapify to make it Max Heap.



Heap after 55 deleted

Max Heap

list of numbers after swapping 55 with 15.

12, 15, 10, 23, **55**, **77**, **82**, **90**

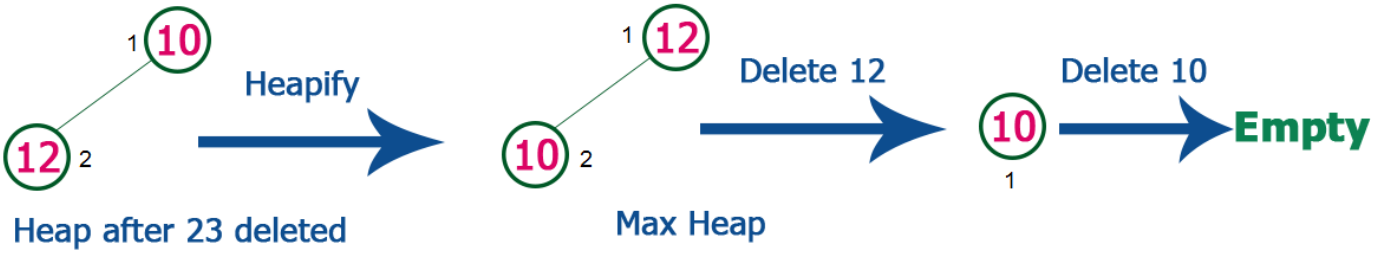
Step 6 - Delete root (**23**) from the Max Heap. To delete root node it needs to be swapped with last node (**12**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 23 with 12.

12. 15. 10. 23. 55. 77. 82. 90

Step 7 - Delete root (**15**) from the Max Heap. To delete root node it needs to be swapped with last node (**10**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after Deleting 15, 12 & 10 from the Max Heap.

10, 12, 15, 23, 55, 77, 82, 90

Whenever Max Heap becomes Empty, the list get sorted in Ascending order

Heapify Method:

1. Maintaining the Heap Property: Heapify is a procedure for manipulating heap Data Structure. It is given an array A and index I into the array. The sub-tree rooted at the children of A [i] are heap but node A [i] itself may probably violate the heap property i.e. A [i] < A [2i] or A [2i+1]. The procedure 'Heapify' manipulates the tree rooted at A [i] so it becomes a heap.

MAX-HEAPIFY (A, i)

1. l ← left [i]
2. r ← right [i]
3. if (l ≤ heap-size[A]) and (A[l] > A [i]) then largest ← l
   else largest ← i
4. If (r ≤ heap-size[A]) and (A[r] > A[largest]) then largest ← r
5. If (largest ≠ i) then swap A[i] ←→ A[largest] and call MAX-HEAPIFY (A, largest)

Building a Heap:

BUILD-MAX-HEAP (A)

1. for i ← |length[A]/2| down to 1
2. do MAX-HEAPIFY(A, i)

Heap-Sort Algorithm:

HEAP-SORT (A)

1. BUILD-MAX-HEAP (A)
2. For i ← length[A] down to 2
   do swap A[1] with A[i]
   Heap-size[A] ← heap-size [A]-1
   MAX-HEAPIFY (A,1)

Analysis: Build max-heap takes **O (n log n)** running time. The Heap Sort algorithm makes a call to 'Build Max-Heap' which we take **O (n log n)** time. The 'Max-Heapify' takes time **O (n log n).** So, **t**he total running time of Heap-Sort is **O (n log n).**

Hashing: Hashing is the transformation of a string of characters into usually a shorter fixed-length value or key that represents the original string.

Hashing is used to index and retrieve items in a database because it is faster to find the item using the shortest hashed key than to find it using the original value. It is also used in many encryption algorithms.

Hashing is a technique in which given key field value is converted into the address of storage location of the record.

Hash Function: Hash Function is used to index the original value or key and then used later each time the data associated with the value or key is to be retrieved. Thus, hashing is always a one-way operation.

Characteristics of Good Hash Function:

1. The hash value is fully determined by the data being hashed.
2. The hash function "uniformly" distributes the data across the entire set of possible hash values.

Some Popular Hash Functions

1. Division Method: This is the easiest method to create a hash function. The hash function can be described as − **h(k) = k mod n**

Here, h(k) is the hash value obtained by dividing the key value k by size of hash table **n** using the remainder. It is best that n is a prime number as that makes sure the keys are distributed with more uniformity.

An example of the Division Method is as follows −

k=1276, n=10
h(1276) = 1276 mod 10
= 6

The hash value obtained is 6. So, the record containing the field value 1276 will be stored at location 6.

A disadvantage of the division method id that consecutive keys map to consecutive hash values in the hash table. This leads to a poor performance.

2. Multiplication Method: The hash function used for the multiplication method is −

$$h(k) = \lfloor n(kA \bmod 1) \rfloor$$

Here, k is the key and A can be any constant value between 0 and 1. Both k and A are multiplied and their fractional part is separated. This is then multiplied with n to get the hash value.

Example – Assume k=123, n=100, A=0.618033
h(123) = ⌊100 (123 * 0.618033 mod 1) ⌋
= ⌊100 (76.018059 mod 1) ⌋
= ⌊100 (0.018059) ⌋
= 1

The hash value obtained is 1. An advantage of the multiplication method is that it can work with any value of A, although some values are believed to be better than others.

3. Mid Square Method: The mid square method is a very good hash function. It involves squaring the value of the key and then extracting the middle r digits as the hash value. The value of r can be decided according to the size of the hash table.

Example − Suppose the hash table has 100 memory locations. So r =**2** because two digits are required to map the key to memory location.

Let k = 40
k*k = 1600
h(40) = 60

The hash value obtained is 60

Hash Tables: A hash table is a data structure that maps keys to values. It uses a hash function to calculate the index for the data key and the key is stored in the index. An example of a hash table is as follows −

The key sequence that needs to be stored in the hash table is −

35, 50, 11, 79, 76, 85

The hash function h(k) used is: h(k) = k mod 10

Using linear probing, the values are stored in the hash table as –

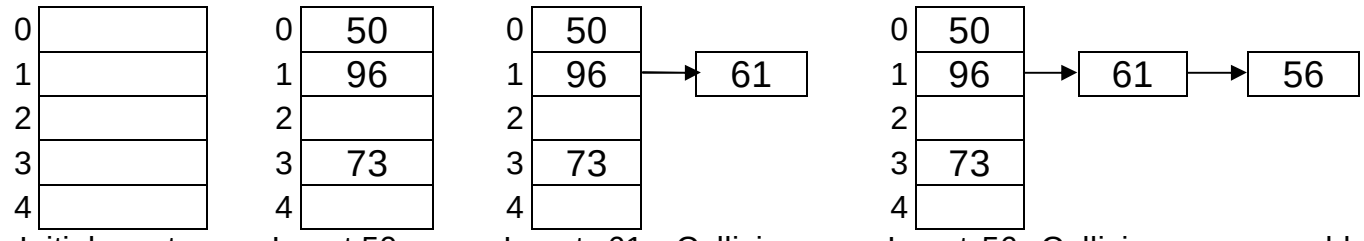
0	50
1	11
2	
3	
4	
5	35
6	76
7	85
8	
9	79

Collision in hashing: Since a hash function gets us a small number for a key which is a big integer or string, there is a possibility that two keys result in the same value. The situation where a newly inserted key maps to an already occupied slot in the hash table is called collision and must be handled using some collision handling technique.

Collisions resolution: There are mainly two methods to handle collision:

1) Chaining
2) Open Addressing

1. Chaining: The idea is to make each cell of hash table point to a linked list of records that have same hash function value. Let us consider a simple hash function as "**key mod 5**" and sequence of keys as 50, 73, 96, 61, 56



Initial empty Table | Insert 50, 73, 96 | Insert 61; Collision occurs, add to chain | Insert 56; Collision occurs, add to chain

Advantages:

1) Simple to implement.
2) Hash table never fills up, we can always add more elements to the chain.

Disadvantages:

1) Cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.
2) Wastage of Space (Some Parts of hash table are never used)
3) If the chain becomes long, then search time can become O(n) in the worst case.
4) Uses extra space for links.

2. Open Addressing: Like chaining, open addressing is a method for handling collisions. In Open Addressing, all elements are stored in the hash table itself. So at any point, the size of the table must be greater than or equal to the total number of keys.

Insert(k): Keep probing until an empty slot is found. Once an empty slot is found, insert k.

Search(k): Keep probing until slot's key doesn't become equal to k or an empty slot is reached.

Delete(k): If we simply delete a key, then the search may fail. So, slots of deleted keys are marked specially as "deleted". The insert can insert an item in a deleted slot, but the search doesn't stop at a deleted slot.

Open Addressing is done in the following ways:

a) Linear Probing
b) Quadratic Probing
c) Double Hashing

a) Linear Probing: In linear probing, we linearly probe for next slot. Suppose a new record R with key k is to be added to the memory table T but that the memory locations with the hash address H (k) is already filled i.e. a collision occurs. We linearly search the next free location to insert k.

Linear probing is simple to implement, but it suffers from an issue known as primary clustering. Long runs of occupied slots build up, increasing the average search time.

Linear probing uses the hash function **h (k, i) = (h' (k) + i) mod m** Where 'm' is the size of hash table and **h' (k) = k mod m** for i=0, 1....m-1.

Example: Consider inserting the keys 50, 74, 97, 85, 56 into a hash table of size m=5 using linear probing, consider the primary hash function is h' (k) = k mod m.

0	
1	
2	
3	
4	

Initial empty Table

0	50
1	
2	97
3	
4	74

Insert 50, 74, 97

0	50
1	85
2	97
3	
4	74

Insert 85; Collision occurs, insert 85 at next free location

0	50
1	85
2	97
3	56
4	74

Insert 56; Collision occurs, insert 56 at next free location

b) Quadratic Probing Suppose a record R with key k has the hash address hash (k) = h then instead of searching the location with addresses h, h+1, and h+ 2...We use has function **$h(k,i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$** to search the location of k,

Where (as in linear probing) h' is an auxiliary hash function c1 and c2 ≠0 are constants and i=0, 1...m-1. The initial position is h' (k); later position probed is offset by the amount that depend in a quadratic manner on the probe number i.

Example: Consider inserting the keys 74, 28, 36, 58, 21, 64 into a hash table of size m = 11 using quadratic probing with c_1=1 and c_2=3. Further consider that the primary hash function is h' (k) = k mod m.

Solution: For Quadratic Probing, we have $h(k, i) = [k \bmod m + c_1 i + c_2 i^2) \bmod m$

Here c_1= 1 and c_2=3

$h(k, i) = [k \bmod m + i + 3i^2] \bmod m$

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	

Initial empty Table

0	
1	
2	
3	36
4	
5	
6	28
7	
8	74
9	
10	

Insert 74, 28, 36

0	
1	
2	
3	36
4	
5	
6	28
7	58
8	74
9	
10	

Insert 58; Collision occurs, insert 85 at $3+1x1+3x1^2$ ie. at 7

0	
1	
2	
3	36
4	
5	
6	28
7	58
8	74
9	
10	21

Insert 21

0	
1	
2	
3	36
4	
5	
6	28
7	58
8	74
9	64
10	21

Insert 64

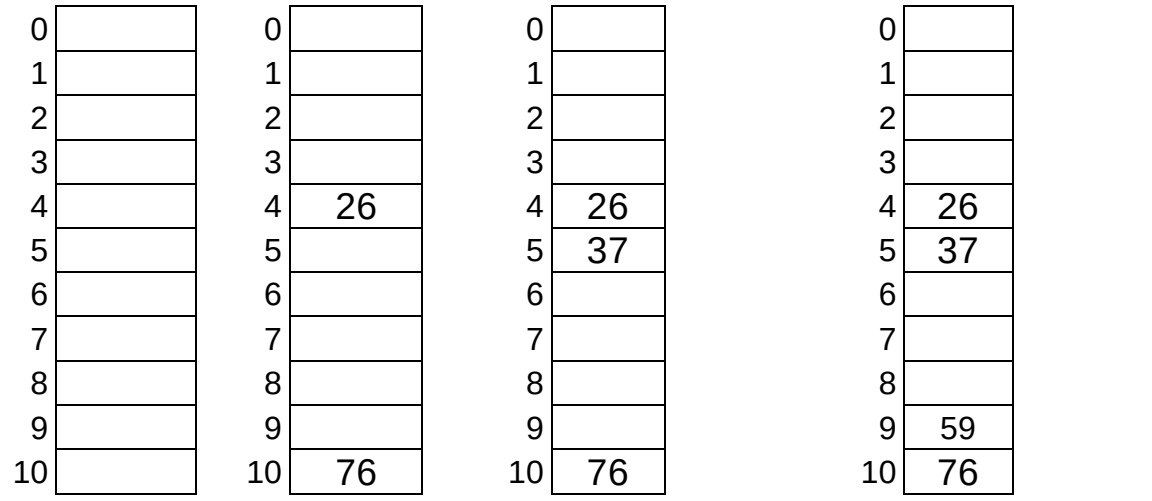
c) Double Hashing: Double hashing uses a hash function of the form

$h(k, i) = (h_1(k) + i\, h_2(k)) \bmod m$ Where h_1 and h_2 are auxiliary hash functions and **m** is the size of the hash table.

$h_1(k) = k \bmod m$ or $h_2(k) = k \bmod m'$. Here m' is slightly less than m (say m-1 or m-2).

Example: Consider inserting the keys 76, 26, 37,59 into a hash table of size m = 11 using double hashing. Consider that the auxiliary hash functions are h_1 (k)=k mod 11 and h_2(k) = k mod 9.

Solution:



Initial empty Table

Insert 76, 26

Insert 37; Collision occurs, insert 37 at 4+1x37%9 i.e. at 5

Insert 59; Collision occurs, insert 59 at 4+1x59%9 i.e. at 9